¿Qué es y para qué sirve JSON?

JSON (*JavaScript Object Notation*) es un formato para el intercambio de datos, básicamente JSON describe los datos con una sintaxis dedicada que se usa para identificar y gestionar los datos.

JSON nació como una **alternativa a XML**, el fácil uso en **javascript** ha generado un gran número de seguidores de esta alternativa. Una de las mayores ventajas que tiene el uso de JSON es que puede ser leído por casi **cualquier lenguaje de programación** (PHP, Java, Phyton,…). Por lo tanto, puede ser usado para el intercambio de información entre distintas tecnologías.

JSON puede representar **seis tipos de valores**:

1. **Objeto:** `{ "nombre": "Ana", "edad": 30 }`
2. **Array:** `[1, 2, 3, 4]`
3. **String (cadena):** `"Hola mundo"`
4. **Number (número):** `42`
5. **Boolean:** `true`
6. **Null:** `null`

Pero en el mundo del desarrollo web y AJAX casi el 100% de las veces se usa para enviar Objetos o Arrays pues la mayoría de las peticiones AJAX consultan bases de datos y la estructura natural de respuesta es un array o un objeto.

Está basado en la notación de objetos de JavaScript y se construye a partir de colecciones de pares nombre/valor (como un objeto JavaScript) y listas ordenadas de valores (como un *array*).

Veamos un sencillo ejemplo de JSON:

```
{
    “mensaje”: "...",
    “parámetros”: {“teléfono”: "...",
                   “codigo_postal”: "...",
                   “fecha_nacimiento”: "..."
                  }
}
```

Los datos JSON son cadenas y deben ser convertidos a un objeto nativo de JavaScript cuando se requiera acceder a dichos datos. Esto no es un problema, dado que JavaScript posee un objeto global JSON que tiene los métodos disponibles para convertir entre ellos.

Diferencias en la declaración de datos en JavaScript y JSon

Array literal en Javascript:

```
var usuario=['juan',26];
```

Como vemos los elementos de un array literal se encierran entre corchetes y los valores contenidos van separados por coma.

Cuando definimos un array literal no le indicamos a cada elemento de que tipo se trata, podemos almacenar cadenas (entre comillas), números, valores lógicos (true,false) y el valor null.

Para acceder a los elementos de un array literal lo hacemos por su nombre y entre corchetes indicamos que elementos queremos acceder:

```
  alert(usuario[0]); //Imprimimos el primer elemento del array
  alert(usuario[1]); //Imprimimos el segundo elemento del array
```

Objeto literal en JavaScript:

```
var persona={
  'nombre':'juan',
  'clave':'xyz',
  'edad':26
};
```

Los objetos literales se definen por medio de pares 'nombre':'valor'

Todo objeto literal tiene un nombre, en el ejemplo persona. Un objeto literal contiene una serie de propiedades con sus respectivos valores. Todas las propiedades del objeto se encuentran encerradas entre llaves. Luego cada propiedad y valor se las separa por dos puntos. Y por último cada propiedad se las separa de las otras propiedades por medio de la coma.

Para acceder a las propiedades del objeto literal lo podemos hacer de dos formas:

```
alert(persona.nombre); //Imprime el valor de la propiedad nombre del
obbjeto persona
```

La segunda forma es indicando la propiedad entre corchetes:

```
alert(persona['nombre']);
```

Además se pueden combinar objetos literales y array literales:

```
var persona={
             'nombre':'juan',
             'edad':22,
             'estudios':['primario','secundario']
            };
```

Luego para acceder a los distintos elementos tenemos:

```
alert(persona.nombre);
```

```
alert(persona.estudios[0]);
alert(persona.estudios[1]);
```

La **sintaxis de JSON** difiere levemente de lo visto anteriormente:

El array se declararía así:

["juan", 26]

Y el objeto:

```
{
  "nombre":"juan",
  "edad":22,
  "estudios":["primario","secundario"]
}
```

Como podemos ver en la sintaxis JSON no aparecen variables, sino directamente indicamos entre llaves o corchetes las propiedades y sus valores.

En JSon solo se admiten comillas dobles y también hemos eliminado el punto y coma después de la llave final. El resto es igual.

```
{
    "marca": "Sony",
    "modelo": "Z",
    "especificaciones": {
        "ram": 1024,
        "resolicion": [1920, 1080],
        "camara": "13Mpx"
    },
    "colores": [
        "rojo",
        "azul",
        "negro"
    ]
}
```

JSON en JavaScript

1. Envío (JS a Cadena JSON)

Cuando quieres enviar un objeto JavaScript al servidor (ej. un formulario complejo), debes convertirlo a una cadena JSON para que pueda viajar a través de HTTP:

Método	Descripción	Ejemplo
JSON.stringify()	Convierte un objeto o valor de JavaScript a una cadena de texto JSON.	`const jsonString = JSON.stringify(miObjeto);`

2. Recepción (Cadena JSON a JS)

Cuando recibes la respuesta JSON del servidor, la respuesta se obtiene mediante la propiedad **responseText** y necesitas convertir la cadena de texto de vuelta a un objeto JavaScript para poder manipular los datos:

Método	Descripción	Ejemplo
JSON.parse()	Analiza la cadena JSON y la transforma en un valor u objeto de JavaScript.	`const datosObj = JSON.parse(xhr.responseText);`

Ejemplo de procesamiento de salida:

JavaScript

```javascript
// 1. Objeto de datos
const usuario = {
    id: 123,
    nombre: "Juan",
    rol: "admin"
};

// 2. Serialización: Convertimos el objeto a cadena JSON
const datosJSON = JSON.stringify(usuario);

// 3. Preparación de AJAX
const xhr = new XMLHttpRequest();
xhr.open("POST", "servidor.php", true);

// Indicamos que enviamos datos de formulario
xhr.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");

xhr.send("dato=" + datosJSON);
```

Ejemplo de procesamiento de entrada:

```javascript
const xhr = new XMLHttpRequest();
xhr.open("GET", "servidor.php?codigo=P01", true);

xhr.onload = function() {
    if (xhr.status === 200) {
        // 1. Recibimos la cadena de texto plana del servidor
        const respuestaTexto = xhr.responseText;

        // 2. Convertimos la cadena JSON de vuelta a un objeto JS
        const objetoDatos = JSON.parse(respuestaTexto);

        // 3. Accedemos a las propiedades del objeto para pintar el DOM
        if (objetoDatos.encontrado) {
            const contenedor = document.getElementById("respuesta");

            // Ejemplo de acceso: objeto.propiedad
            contenedor.textContent = `Producto: ${objetoDatos.producto.nombre}
                                      - Precio: ${objetoDatos.producto.precio}€`;
        }
    }
};
```

El servidor envía: {"encontrado":true,"producto":{"nombre":"Monitor","precio":350}}

JSON en PHP

PHP tiene funciones nativas y muy sencillas para manejar **JSON**, permitiendo la fácil conversión entre cadenas JSON y las estructuras de datos nativas de PHP (arrays y objetos).

1. Envío (PHP a Cadena JSON)

La función json_encode() toma una variable de PHP (generalmente un array o un objeto) y la convierte en una cadena de texto JSON adecuada para ser enviada al navegador o a otra aplicación.

Método	Descripción	Ejemplo
json_encode()	Convierte una variable de PHP (array u objeto) en una cadena de texto JSON.	`$json = json_encode($miVariable);`

2. Recepción (Cadena JSON a estructura PHP)

Cuando recibes datos desde JavaScript (enviados con JSON.stringify()), PHP los recibe como una cadena de texto que debe ser analizada para poder usarla.

Método	Descripción	Ejemplo
json_decode()	Toma una cadena JSON y la convierte en un objeto o un array de PHP.	`$datos = json_decode($cadenaJson);`

La función json_decode tiene la siguiente estructura:

json_decode(string $json, bool $assoc = false)

El **parámetro $assoc** indica lo siguiente:

- Si $assoc es **false** (por defecto), PHP devuelve un **objeto genérico** (stdClass).
- Si $assoc es **true**, PHP devuelve un **array asociativo**.

```php
<?php
// Supongamos que esta es la cadena JSON recibida
$json_input = '{"nombre":"Ana", "edad":30}';

// Opción A: Decodificar a objeto PHP (por defecto)
$objeto_php = json_decode($json_input);
echo "Nombre (Objeto): " . $objeto_php->nombre; // Salida: Ana

// Opción B: Decodificar a array asociativo
$array_php = json_decode($json_input, true);
echo "Edad (Array): " . $array_php['edad']; // Salida: 30
?>
```

Ejemplo de procesamiento de salida:

```php
<?php
// 1. Preparamos los datos (Array asociativo o stdClass)
$producto = [
    "id" => "P01",
    "nombre" => "Monitor",
    "precio" => 150.99
];

// 2. Convertimos a JSON
$json_output = json_encode($producto);

// 3. Importante: Informar al navegador del tipo de contenido
header('Content-Type: application/json');

// 4. Enviamos la cadena
echo $json_output;
?>
```

Ejemplo de procesamiento de entrada:

```php
<?php
// 1. Recogemos la cadena JSON desde el parámetro 'dato'
$json_recibido = $_REQUEST['dato'];

// 2. Convertimos la cadena de texto a un objeto PHP (stdClass)
$objeto_php = json_decode($json_recibido);

// 3. Acceso a los datos (Sintaxis de objeto)
if ($objeto_php) {
    $nombre = $objeto_php->nombre;
    $email  = $objeto_php->email;
}
?>
```

RESUMEN FUNCIONES

Acción	En el Cliente (JavaScript)	En el Servidor (PHP)
Convertir a Texto (Para enviar)	**JSON.stringify(obj)** conviete de JS a JSon	**json_encode($var)** conviete de PHP a JSon
Convertir a Datos (Al recibir)	**JSON.parse(texto)** conviete de JSon a JS	**json_decode($texto)** conviete de JSon a PHP